

MI NIR API

Version 0.1

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
0.1	Initial release	08/20/2021
0.2	Remove vis/contrast/saturation level	01/04/2022

TABLE OF CONTENTS

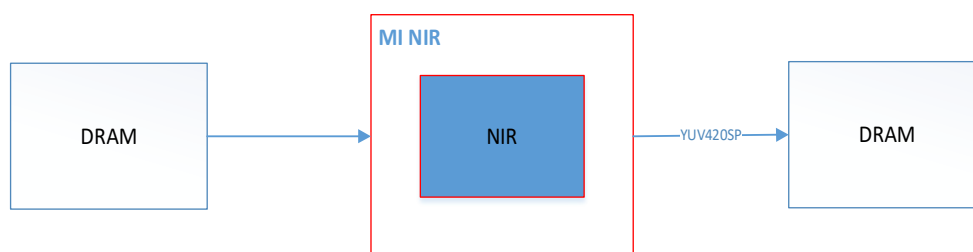
REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. API 参考.....	1
1.1. 概述.....	1
1.2. 流程图.....	1
1.3. 关键说明.....	1
2. 功能模块 API.....	3
2.1. MI_NIR_CreateDevice.....	3
2.2. MI_NIR_DestroyDevice.....	4
2.3. MI_NIR_CreateChannel.....	5
2.4. MI_NIR_DestroyChannel.....	5
2.5. MI_NIR_SetChnParam.....	6
2.6. MI_NIR_GetChnParam.....	7
2.7. MI_NIR_StartChannel.....	8
2.8. MI_NIR_StopChannel.....	8
3. 数据类型.....	10
3.1. 数据结构列表.....	10
3.2. MI_NIR_DEV.....	10
3.3. MI_NIR_CAHNNEL.....	10
3.4. MI_NIR_DevAttr_t.....	11
3.5. MI_NIR_Mode_e.....	11
3.6. MI_NIR_ChnAttr_t.....	12
3.7. MI_NIR_ChnParam_t.....	12
4. 错误码.....	13

1. API 参考

1.1. 概述

NIR 是 SStar 通过 VIS 和 NIR 采集的图像融合成新的图像，在黑暗环境下，既可以输出高质量彩色画面，又可以减少光污染。

1.2. 流程图



输入图像在 **dram** 中 的格式见下面关键字的说明。

1.3. 关键说明

Input dram layout:



VIS:

Visible 可见光谱

NIR:

near-infrared 近红外

Device:

NIR 硬件设备

Channel:

NIR 设备的通道，各个通道分时复用 NIR 硬件

输入图像格式:

YUV420SP

输出图像格式：
YUV420SP

输入分辨率：

Muffin：

最大支持 2688 x 1520

支持输入图像宽度 8Pixel 对齐，Stride 16 对齐

输出分辨率：

跟输入分辨率一致

2. 功能模块 API

如下是 MI NIR 模块的用户接口。

API 名	功能
系统功能类	
MI_NIR_CreateDevice	创建 NIR Device 设备
MI_NIR_DestroyDevice	销毁 NIR Device 设备
MI_NIR_CreateChannel	创建 NIR Channel 通道
MI_NIR_DestroyChannel	销毁 NIR Channel 通道
MI_NIR_StartChannel	启用 NIR Channel 通道
MI_NIR_StopChannel	禁用 NIR Channel 通道
MI_NIR_SetChnParam	配置 NIR Channel 通道属性
MI_NIR_GetChnParam	获取 NIR Channel 通道属性

2.1. MI_NIR_CreateDevice

➤ 功能

创建 NIR 设备。

➤ 语法

MI_S32 MI_NIR_CreateDevice(MI_NIR_DEV DevId , [MI_NIR_DevAttr_t](#) *pstDevAttr);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID, 对应 NIR Device ID	输入
pstDevAttr	Device 设备属性，静态参数	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败，参照[错误码](#)。

- 依赖
 - 头文件: mi_nir.h
 - 库文件: libmi_nir.a / libmi_nir.so

- ※ 注意
 - 目前只支持 DevId =0;

- 举例
NIR 初始化设置, 以及退出范例

```
MI_NIR_DEV  NIRDevId =0;
MI_NIR_CHANNEL  NIRChnId =0;
MI_NIR_DevAttr_t  stDevAttr = {0};
MI_NIR_ChnAttr_t  stChnAttr = {};

stChnAttr.eMode= E_MI_NIR_MODE_NORMAL ;//设置 CHN 动态参数

MI_NIR_CreateDevice(NIRDevId, &stDevAttr);
MI_NIR_CreateChannel(NIRDevId, NIRChnId, &stChnAttr);
MI_NIR_StartChannel(NIRDevId, NIRChnId);

/*销毁通道*/
MI_NIR_StopChannel(NIRDevId, NIRChnId);
MI_NIR_DestroyChannel(NIRDevId,NIRChnId);
MI_NIR_DestroyDevice(NIRDevId)
```

- 相关的主题
MI_NIR_DestroyDevice

2.2. MI_NIR_DestroyDevice

- 功能
销毁 NIR 设备。
- 语法
MI_S32 MI_NIR_DestroyDevice(MI_NIR_DEV DevId);

- 形参

参数名称	描述	输入/输出
DevId	Device ID	输入

- 返回值
 - MI_SUCCESS(0) 成功。
 - 非 0 失败, 参照[错误码](#)。
- 依赖

- 头文件: mi_nir.h
- 库文件: libmi_nir.a / libmi_nir.so

※ 注意

- 目前只支持 DevId =0;

➤ 举例

见 MI_NIR_CreateDevice 举例

➤ 相关的主题

MI_NIR_CreateDevice

2.3. MI_NIR_CreateChannel

➤ 功能

创建 NIR Channel 通道。

➤ 语法

MI_S32 MI_NIR_CreateChannel(MI_NIR_DEV DevId , MI_NIR_CHANNEL ChnId, [MI_NIR_ChannelAttr_t](#) *pstChAttr);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入
pstChAttr	Channel 属性	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败, 参照[错误码](#)。

➤ 依赖

- 头文件: mi_nir.h
- 库文件: libmi_nir.a / libmi_nir.so

※ 注意

- 目前只支持 ChnId =0;
- 需要在 MI_NIR_CreateDevice 之后调用

➤ 举例

见 MI_NIR_CreateDevice 举例

➤ 相关的主题

MI_NIR_DestroyChannel

2.4. MI_NIR_DestroyChannel

➤ 功能

销毁 NIR Channel 通道。

➤ 语法

MI_S32 MI_NIR_DestroyChannel(MI_NIR_DEV DevId , MI_NIR_CHANNEL ChnId);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_nir.h
- 库文件：libmi_nir.a / libmi_nir.so

※ 注意

- 目前只支持 ChnId =0;
- 需要在 MI_NIR_CreateDevice 之后调用

➤ 举例

见 MI_NIR_CreateDevice 举例

➤ 相关的主题

MI_NIR_CreateChannel

2.5. MI_NIR_SetChnParam

➤ 功能

设置 NIR Channel 动态参数

➤ 语法

MI_S32 MI_NIR_SetChnParam(MI_NIR_DEV DevId , MI_NIR_CHANNEL ChnId, MI_NIR_ChnParam_t *pstChnParam);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入
pstChnParam	Channel 动态参数	输入

➤ 返回值

- MI_SUCCESS(0) 成功。

- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_nir.h
- 库文件：libmi_nir.a / libmi_nir.so

※ 注意

- 目前只支持 ChnId =0;
- Device 和 channel 创建之后调用

➤ 举例

```
MI_NIR_DEV NIRDevId =0;
MI_NIR_CHANNEL NIRChnId =0;

MI_NIR_ChnParam_t stChnParam = {};

MI_NIR_GetChnParam(NIRDevId , NIRChnId,& stChnParam);

stChnParam.eMode= E_MI_NIR_MODE_NORMAL;//设置 CHN 动态参数

MI_NIR_SetChnParam(NIRDevId , NIRChnId, & stChnParam);
```

➤ 相关的主题

MI_NIR_GetChnParam

2.6. MI_NIR_GetChnParam

➤ 功能

获取 NIR Channel 动态参数

➤ 语法

```
MI_S32 MI_NIR_GetChnParam(MI_NIR_DEV DevId , MI_NIR_CHANNEL ChnId, MI_NIR_ChnParam
*pstChnParam);
```

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入
pstChnParam	Channel 动态参数	输出

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_nir.h
- 库文件：libmi_nir.a / libmi_nir.so

※ 注意

N/A

➤ 举例

见 MI_NIR_SetChnParam 举例

➤ 相关的主题

MI_NIR_SetChnParam

2.7. MI_NIR_StartChannel

➤ 功能

启动 NIR Channel。

➤ 语法

MI_S32 MI_NIR_StartChannel(MI_NIR_DEV DevId , MI_NIR_CHANNEL ChnId);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_nir.h
- 库文件：libmi_nir.a / libmi_nir.so

※ 注意

- 目前只支持 ChnId =0;
- Device 和 channel 创建之后调用

➤ 举例

见 MI_NIR_CreateDevice 举例

➤ 相关的主题

MI_NIR_StopChannel

2.8. MI_NIR_StopChannel

➤ 功能

禁用 NIR Channel 通道。

➤ 语法

MI_S32 MI_NIR_StopChannel(MI_NIR_DEV DevId , MI_NIR_CHANNEL ChnId);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_nir.h
- 库文件：libmi_nir.a / libmi_nir.so

※ 注意

N/A

➤ 举例

见 MI_NIR_CreateDevice 举例

➤ 相关的主题

MI_NIR_StartChannel

3. 数据类型

3.1. 数据结构列表

相关数据类型、数据结构定义如下：

MI_NIR_DEV	定义 NIR Device 的类型
MI_NIR_CHANNEL	定义 NIR Channel 的类型
MI_NIR_DevAttr_t	定义 NIR Device 属性
MI_NIR_Mode_e	定义 NIR 模式设定
MI_NIR_ChnAttr_t	定义 NIR Channel 的属性。
MI_NIR_ChnParam_t	定义 NIR Channel 的动态参数

3.2. MI_NIR_DEV

➤ 说明

定义 NIR Device 的类型。

- 定义
typedef MI_U32 MI_NIR_DEV

- ※ 注意事项
N/A

- 相关数据类型及接口
N/A

3.3. MI_NIR_CHANNEL

- 说明
定义 NIR channel 的类型。

- 定义
typedef MI_U32 MI_NIR_CHANNEL

- ※ 注意事项
N/A

- 相关数据类型及接口
N/A

3.4. MI_NIR_DevAttr_t

- 说明
定义设备 Device 的属性。

- 定义
typedef struct MI_NIR_DevAttr_s
{
 MI_U32 u32Reserved;
} MI_NIR_DevAttr_t;

- 成员

成员名称	描述
u32Reserved	保留数据

- ※ 注意事项
N/A

- 相关数据类型及接口
N/A

3.5. MI_NIR_Mode_e

- 说明
定义 NIR 模式，表示 NIR 数据处理方式。

➤ 定义

```
typedef enum
{
    E_MI_NIR_MODE_NORMAL,          //normal
    E_MI_NIR_MODE_BYPASS_VIS,      //bypass visible data
    E_MI_NIR_MODE_BYPASS_NIR,      // bypass near-infrared data
    E_MI_NIR_MODE_INVALID,

} MI_NIR_Mode_e;
```

※ 注意事项

N/A

➤ 相关数据类型及接口

MI_NIR_ChnParam_t;

3.6. MI_NIR_ChnAttr_t

➤ 说明

定义 NIR Channel 的属性。

➤ 定义

```
typedef struct MI_NIR_ChnAttr_s
{
    MI_NIR_Mode_e eMode;
} MI_NIR_ChnAttr_t;
```

➤ 成员

成员名称	描述
eMode	模式设定

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.7. MI_NIR_ChnParam_t

➤ 说明

定义 NIR Channel 的动态参数。

➤ 定义

```
typedef struct MI_NIR_ChnParam_s
{
    MI_NIR_Mode_e eMode;
} MI_NIR_ChnParam_t;
```

➤ 成员

成员名称	描述
eMode	模式设定

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

4. 错误码

MI NIR 模块 错误码如下表所示

错误码	宏定义	描述
0	MI_NIR_RET_SUCCESS	成功
0xA0272001	MI_ERR_NIR_MOD_NOT_INIT	模块未完成初始化
0xA0272002	MI_ERR_NIR_DEV_INVALID	Device 超出合法范围
0xA0272003	MI_ERR_NIR_NORESOURCE	NIR 没有足够资源
0xA0272004	MI_ERR_NIR_CHN_INVALID	Channel 超出合法范围
0xA0272005	MI_ERR_NIR_TIMEOUT	NIR 设备运行超时
0xA0272006	MI_ERR_NIR_NOMEM	分配内存失败
0xA0272007	MI_ERR_NIR_NOTSUPPORT	操作不支持
0xA0272008	MI_ERR_NIR_BUSY	Channel 系统忙
0xA0272009	MI_ERR_NIR_NOTREADY	设备未完成初始化
0xA027200A	MI_ERR_NIR_NULL_PTR	输入参数空指针错误